

Threads – Java

- Interrupt
- Sleep
- Join

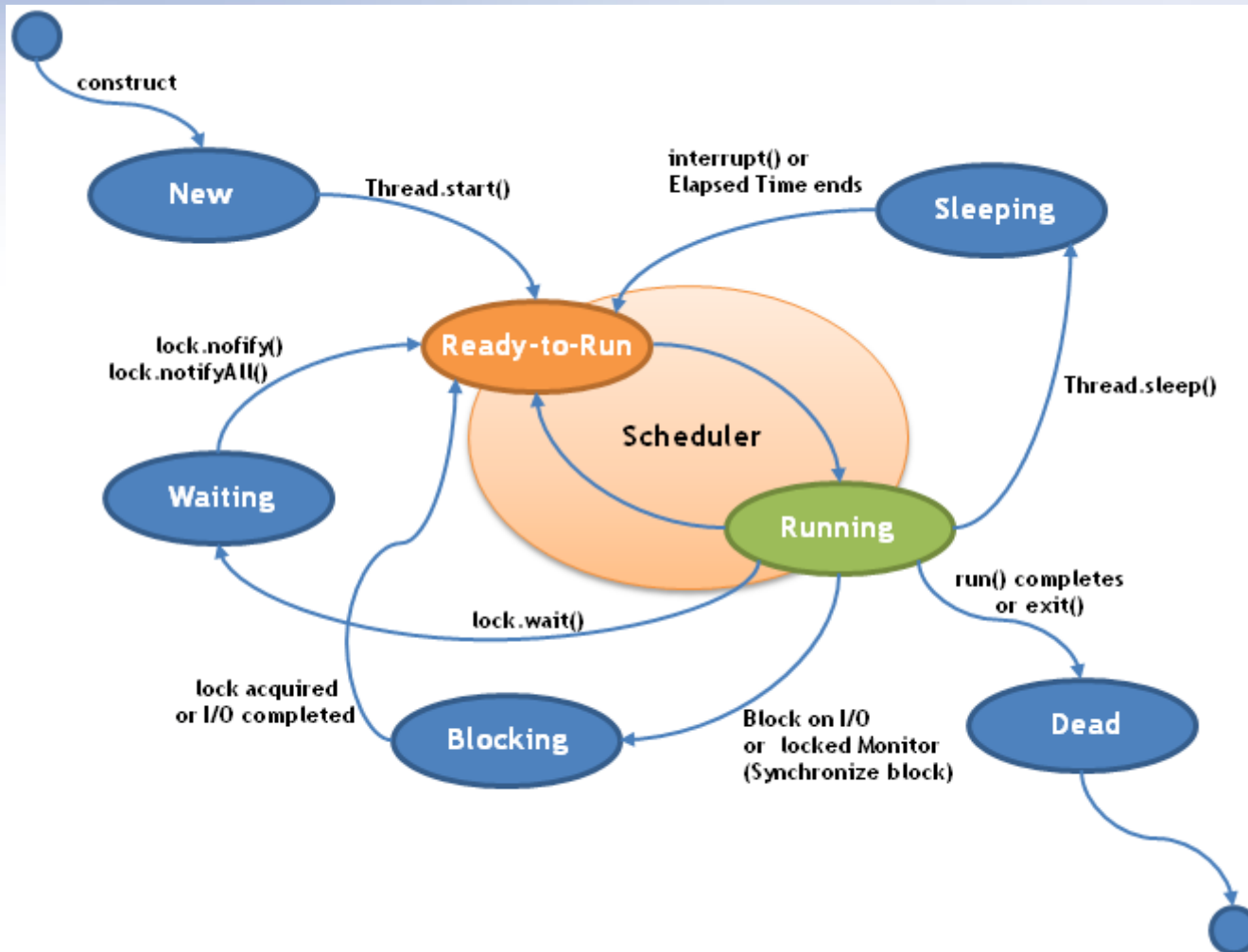
Programacion Concurrente y Paralela 2026

Dr. Ing. Ventre, Luis O.

Ing. Ludemann, Mauricio.

Ing. Carranza, Agustin.

Threads



Interrupting A Thread

- Un programa JAVA, con más de un hilo en ejecución solo finaliza, cuando todos sus hilos (no daemons) terminan su ejecución.
- Puede ser necesario finalizar la ejecución de un Hilo. Por ej. Si queremos terminar un programa o finalizar la tarea que el hilo está haciendo.
- Java provee un mecanismo para indicarle al hilo **nuestra voluntad (gentil)** de detenerlo.
- Una peculiaridad del mecanismo, es que el hilo puede ignorar la petición y continuar trabajando.

Interrupting A Thread

Ejemplo

- Se realizara un programa que crea un hilo y 10 segundos después solicita su fin usando el mecanismo de interrupción.

```

Main.java  PrimeGenerator.java
1  package Pkt;
2
3  public class Main {
4
5      public static void main(String[] args) {
6          Thread task = new PrimeGenerator();
7          task.start();
8
9          try {
10             Thread.sleep(10000);
11         }
12         catch (InterruptedException e) {
13             e.printStackTrace();
14         }
15
16         task.interrupt();
17     }
18 }
19
```

Interrupting A Thread

Ejemplo

```
package Pkt;

public class PrimeGenerator extends Thread {

    @Override
    public void run() {

        long number = 1L;
        while(true){

            if(isPrime(number)){
                System.out.println("Number" + number + " is Prime");
            }

            if (isInterrupted()){
                System.out.printf("The prime generator has been interrupted\n");
                return;
            }
            number++;
        }
    }

    private boolean isPrime(long number) {
        if(number<=2)
            return true;

        for(long i=2;i<number;i++)
        {
            if((number%i)==0)
                return false;
        }
        return true;
    }
}
```

Interrupting A Thread

Ejemplo

- Salida por consola

```
Number148199 is Prime
Number148201 is Prime
Number148207 is Prime
Number148229 is Prime
Number148243 is Prime
Number148249 is Prime
Number148279 is Prime
Number148301 is Prime
Number148303 is Prime
Number148331 is Prime
Number148339 is Prime
The prime generator has been interrupted
```

Interrupting A Thread

- Ver en la IDE el ejemplo de la version 2 del cookbook. Tiene impresión de flags de estado del hilo.

Console

<terminated> Main (52) [Java Application] C:\Program Files\Java\jre-9.0.4\bin\javaw.exe (26 mar. 2019 18:32:39)

Number 111227 is Prime

Number 111229 is Prime

Number 111253 is Prime

Number 111263 is Prime

Number 111269 is Prime

Number 111271 is Prime

Main: Status of the Thread: RUNNABLE

Main: isInterrupted: true

Main: isAlive: true

Number 111301 is Prime

The Prime Generator has been Interrupted

Interrupting A Thread

- Que sucede si se coloca un sleep, luego del interrupt en el main, antes de imprimir los valores de los flags?

Console

<terminated> Main (52) [Java Application] C:\Program Files\Java\jre-9.0.4\bin\javaw.exe (26 mar. 2019 18:33:30)

```
Number 111977 is Prime
Number 111997 is Prime
Number 112019 is Prime
Number 112031 is Prime
Number 112061 is Prime
Number 112067 is Prime
Number 112069 is Prime
The Prime Generator has been Interrupted
Main: Status of the Thread: TERMINATED
Main: isInterrupted: false
Main: isAlive: false
```


Interrupting A Thread

- El mecanismo visto para la interrupción de un hilo es adecuado para programas simples. Sin recursividad ni complejidad en cantidad de métodos.
- Java provee la interrupción por excepción **InterruptedException**, esta interrupción puede ser lanzada al detectar la interrupción del hilo y hacer el catch desde el método `run()`.

Interrupting A Thread

- El siguiente ejercicio implementará un hilo que busca archivos con un determinado nombre en un directorio y sus subdirectorios.
- Crear una clase llamada FileSearch que implemente la interfaz runnable.
- Declarar dos atributos privados, uno para el nombre del archivo y el otro para el directorio donde comenzar a buscar.

Interrupting A Thread

- Implemente el método `run()`, la cual chequea si el atributo `fileName` es un directorio, y si lo es llama al método `directoryProcess()`.
 - Durante la ejecución de este método, puede ocurrir una excepción por Interrupción (`InterruptedException`) por lo que debemos implementar `catch` correspondiente.
- Implementar el método `directoryProcess()`. En cada llamado al método hará una recursión llamando al método `fileProcess()`.

Interrupting A Thread

- Luego de procesar todos los archivos y directorios el hilo chequea si ha sido interrumpido y lanza la excepción correspondiente.
- Implementar el método `fileProcess`, este método compara el archivo buscado con cada uno, si son iguales, imprime un mensaje en la consola. Luego el hilo chequea si ha sido interrumpido y lanza la excepción correspondiente.

Interrupting A Thread

- Implemente la clase Main.
- Cree e inicialice un objeto de la clase FileSearch.
- Comience la ejecución del hilo.
- Espere 10 segundos
- Interrumpa el hilo.
- Ejecute y vea el resultado.

Interrupting A Thread

- Clase Main

```
*Main.java  FileSearch.java
1 package Pkt;
2
3 import java.util.concurrent.TimeUnit;
4
5 public class Main {
6
7     public static void main(String[] args) {
8
9         FileSearch buscador = new FileSearch("C:\\", "log.txt");
10        Thread hilo = new Thread(buscador);
11
12        hilo.start(); // si pongo run, el hilo nunca se detiene porque?
13
14        try{
15            TimeUnit.SECONDS.sleep(10);
16        } catch (InterruptedException e){
17            e.printStackTrace();
18        }
19
20        hilo.interrupt();
21    }
22
23 }
```

Interrupting A Thread

- Clase FileSearch

```
import java.io.File;

public class FileSearch implements Runnable {

    private String initPath;
    private String fileName;

    public FileSearch(String initPath, String fileName) {
        super();
        this.initPath = initPath;
        this.fileName = fileName;
    }

    public void run() {
        File file=new File(initPath);
        if(file.isDirectory()){
            try{
                directoryProcess(file);
            }catch (InterruptedException e){
                System.out.printf("%s: La búsqueda ha sido interrumpida ",
                                Thread.currentThread().getName());

            }
        }
    }
}
```

Interrupting A Thread

- Método directoryProcess

```
private void directoryProcess(File file) throws InterruptedException {  
  
    File list[]=file.listFiles();  
    if(list != null){  
        for(int i=0;i<list.length;i++){  
            if(list[i].isDirectory()){  
                directoryProcess(list[i]);  
            }else{  
                fileProcess(list[i]);  
            }  
        }  
    }  
  
    if(Thread.interrupted()){  
        throw new InterruptedException();  
    }  
}
```


Interrupting A Thread

- Método fileProcess

```
private void fileProcess(File file) throws InterruptedException {  
    if (file.getName().equals(fileName)) {  
        System.out.printf("%s : %s\n", Thread.currentThread().getName(),  
                           file.getAbsolutePath());  
    }  
  
    if(Thread.interrupted())  
    {throw new InterruptedException();  
    }  
}
```

Interrupting A Thread

- Argumente porque al modificar el código y ejecutar la instrucción `run()` desde el `main` en lugar de `start()`, el buscador SI encuentra el archivo buscado.
- Y al ejecutar la instrucción `start()` no la encuentra.
- Cual puede ser el posible problema?
- Alguna solución?

Sleeping & Resuming a Thread

- En diferentes ocasiones es necesario **suspender la ejecución de un hilo por un determinado tiempo.**
- Por ej. un hilo que lee un sensor, y luego durante un minuto no realiza tarea alguna.
- Puede utilizar el metodo **sleep()** de la clase Thread para este propósito.

Sleeping & Resuming a Thread

- Este método recibe un entero como argumento, que representa el número en **milisegundos** que el hilo suspende su ejecución.
- Cuando el tiempo de sleep finaliza, el hilo continúa con la ejecución de la instrucción siguiente al sleep(). Cuando la JVM le asigne cpu time.
- *Otra posibilidad es utilizar el método sleep() del enumerado TimeUnit.*

Sleeping & Resuming a Thread

Ejemplo

- El siguiente programa utiliza el método `sleep()` para mostrar por la consola la fecha a cada segundo.

```
public class FileClock implements Runnable {
```

Sleeping & Resuming a Thread

```
public void run() {  
    for(int i=0;i<10;i++){  
        System.out.printf("%s\n", new Date());  
        try{  
            TimeUnit.SECONDS.sleep(1);  
        }catch (InterruptedException e){  
            System.out.printf("The FileClock has been interrupted");  
            return;  
        }  
    }  
}
```

Sleeping & Resuming a Thread

```
public class FileMain {  
  
    public static void main(String[] args) {  
  
        FileClock clock= new FileClock();  
        Thread thread=new Thread(clock);  
  
        thread.start();  
  
        try{  
            TimeUnit.SECONDS.sleep(5);  
        }catch (InterruptedException e){  
            e.printStackTrace();  
        }  
  
        thread.interrupt();  
    }  
}
```

Sleeping & Resuming a Thread

- En el ejercicio anterior, al ejecutar **interrupt()** el hilo finalizaba.
- En este ejercicio al ejecutar **interrupt()** que sucede?
- Por qué?
- Comprende la gentileza y la diferencia con el metodo deprecado **thread.stop()**.

Waiting for the finalization

- En algunas situaciones, debemos esperar a que un hilo finalice su ejecución.
 - Por ejemplo: un programa que comience inicializando los recursos antes de comenzar con el resto de la ejecución.
- Para este propósito podemos usar el método **join()** de la clase Thread.
 - Cuando llamamos este método usando un objeto Thread, suspende su ejecución hasta que finalice la ejecución del Thread llamado.

Waiting for the finalization

Ejemplo

- Supongamos que en una clase hay diferentes grupos.
- Nuestro programa debe crear un hilo para administrar la impresion de mensajes de cada grupo
- Desde main se crean los hilos, y se asignan los grupos como objetos que implementan la interfaz runnable.

Waiting for the finalization

- Es deseado que los mensajes se impriman de manera ordenada.
- Y el main debe esperar que cada grupo haya impreso sus mensajes antes de comenzar con el siguiente.

Waiting for the finalization

Main.java Grupo.java

```
3 public class Main {
4
5     public static void main(String[] args) throws InterruptedException {
6         // TODO Auto-generated method stub
7
8         Thread teapots=new Thread(new Grupo("Teapots"));
9         Thread venThread=new Thread(new Grupo("VenThread"));
10        Thread giA=new Thread(new Grupo("GiA"));
11
12        teapots.start();
13        teapots.join();
14
15        venThread.start();
16        venThread.join();
17
18        giA.start();
19        giA.join();
20
21        //teapots.join();
22        //venThread.join();
23        //giA.join();
24
25
26        System.out.printf("Main: Se finalizo la impresion de mensajes %s\n", new Date());
```

Waiting for the finalization

```
Main.java  Grupo.java ✕
1
2 public class Grupo implements Runnable {
3
4     String mensaje;
5
6     public Grupo(String nombre){
7
8         mensaje = "Hola, somos el grupo " + nombre + " y este es nuestro mensaje numero: ";
9
10    }
11    @Override
12    public void run(){
13        // TODO Auto-generated method stub
14
15        for (int i=1; i<100;i++){
16            String msj = mensaje + i;
17            System.out.println(msj);
18        }
19    }
20
21 }
22
23 }
24
```

Waiting for the finalization

- Que sucederia si comentamos el primer join?.
- Que sucederia si comentamos todos los join, y los colocamos luego de todos los starts?
- Que sucederia si no ponemos joins?

Waiting for the finalization

Ejemplo

- Crear una clase llamada **DataSourcesLoader**, la misma debe implementar la interfaz runnable.
- Implemente el método run(), este escribe un mensaje indicando que comienza su ejecución; luego se duerme por 4 segundos, y escribe otro mensaje para indicar que finalizó su ejecución.
- Crear una clase llamada **NetworkConnectionsLoader**, la misma debe implementar la interfaz runnable.
- El método run debe ser igual al anterior pero 6 segundos

Waiting for the finalization

- Crear la clase main.
 - Crear un objeto de cada clase anterior.
 - Crear dos Threads y lanzar la ejecución.
- Desde el main, deberá esperar la finalización de ambos hilos. Utilizando el método `join()`. Este método puede lanzar una `InterruptedException` por lo que se debe incluir `try catch`.
- Imprimir un mensaje de fin de programa. Ejecutar!

Waiting for the finalization

```
import java.util.Date;

public class Main {

    public static void main(String[] args) {
        DataSourceLoader dsLoader = new DataSourceLoader();
        Thread thread1 = new Thread(dsLoader, "DataSourceThread");//Tipo y nombre del thread creado

        NetworkConnectionsLoader ntLoader = new NetworkConnectionsLoader();
        Thread thread2 = new Thread(ntLoader, "NetworkConnectionsLoader Thread");

        thread1.start();
        thread2.start();

        try {
            thread1.join(); //es lo mismo que hacer join solo con el segundo
            thread2.join(); //aca el join hace que el main detenga su ejecucion hasta que finalice
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.printf("Main: Configuration has been loaded: %s\n", new Date());
    }
}
```

Waiting for the finalization

```
⊕ import java.util.Date;

public class DataSourcesLoader implements Runnable {

    ⊖ @Override
    public void run() {
        System.out.printf("Beginning data sources loading: %s\n", new Date());
        try {
            TimeUnit.SECONDS.sleep(4);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.printf("Data sources loading has finished: %s\n", new Date());
    }
}
```

Waiting for the finalization

```
import java.util.Date;

public class NetworkConnectionsLoader implements Runnable {

    @Override
    public void run() {
        System.out.printf("Beginning data sources loading: %s\n", new
            Date());
        try {
            TimeUnit.SECONDS.sleep(9);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.printf("Data sources loading has finished: %s\n", new Date());
    }
}
```

Waiting for the finalization

 Console 

```
<terminated> Main (3) [Java Application] C:\Program Files\Java\jre7\bin\javaw.exe (4 de abr. de 2017 4:51:02 p. m.)  
Beginning data sources loading: Tue Apr 04 16:51:02 ART 2017  
Beginning Network Connections sources loading: Tue Apr 04 16:51:02 ART 2017  
Data sources loading has finished: Tue Apr 04 16:51:06 ART 2017  
NetWork Connections sources loading has finished: Tue Apr 04 16:51:11 ART 2017  
Main: Configuration has been loaded: Tue Apr 04 16:51:11 ART 2017
```